

PLAY IT FORWARD

A Public Arcade Machine for Charitable
Fundraising

Prepared By: Neo Maxwell Tartan
Started on: August 2026

1. Introduction

1.1. Background

This document lays out the concept and technical plan for an arcade machine designed to be placed in public locations such as malls, community centres, or event spaces. Visitors pay a small fee to play, and the money collected is set aside and donated to a chosen public or charitable cause regularly.

The idea sits at the intersection of three interests: embedded electronics and IoT, game design, and using engineering skills for social impact rather than only for personal projects or competitions. It is meant to move through a clear sequence of build phases rather than a fixed calendar, with a target of reaching a working pilot within roughly 1 to 1.5 years (Section 3).

1.2. Why This Project?

1. An arcade cabinet is a passive fundraising tool. Once installed, it keeps generating small donations without needing an ongoing ask, unlike a one-time charity drive.
2. It combines several existing skill areas (embedded systems, sensors, basic game development, and system design) into one coherent build, rather than treating them as separate hobbies.
3. A physical, public-facing machine forces real engineering constraints that a purely academic or competition project does not: durability, tamper resistance, power reliability, and unattended operation.
4. It creates a concrete, demonstrable project (design, build, deploy, report on funds raised) that has value beyond a single competition cycle.

1.3. Core Concept

A standalone arcade cabinet houses a small computer running one or more games, a control panel (joystick and buttons), a QRIS-based cashless payment flow, and a logging layer that records how much has been collected. The cabinet is hosted at a partner venue such as a shopping mall or school event under some form of agreement, and funds raised are periodically withdrawn and donated, with the amounts reported publicly for transparency.

2. Technical Planning

2.1. System Overview

The machine can be thought of as four subsystems that need to work together:

1. Cabinet/frame: the physical wooden housing, control panel, and mounting for all electronics.
2. Compute: the device that actually runs the game and drives the screen.
3. Payment & logging: software for payment through QRIS and automatically logs payment.
4. Software: the game itself, the platform it runs on, and the reporting layer that turns raw play/payment data into a transparent donation record.

2.2. Cabinet / Frame (Kerangka)

The frame is the one part of this project best suited for a local carpenter (tukang kayu) rather than a DIY build, since a plumb, rigid cabinet that survives public use is mostly a woodworking and joinery problem, not an electronics one. This eliminates the need for external futile effort, when it should be spent on more demanding and important matters.

1. Material: 25mm thick plywood for structural panels. MDF is cheaper and easier to cut cleanly but is heavier and less impact-resistant than plywood, which is worth discussing with the carpenter given the machine will sit unsupervised in public.
2. Reference standard upright arcade cabinet dimensions as a starting point (roughly 60–65cm wide, 75–80cm deep, 170–180cm tall) and adjust for the chosen screen size and coin door.
3. Design for serviceability: a lockable rear or side access panel so the computer, coin box, and wiring can be reached without disassembling the cabinet.
4. Ventilation: cut and screen vents near the compute unit so a Pi does not overheat inside a closed wooden box.
5. Finish: a laminate or painted finish that is easy to wipe down and resists scratching/graffiti in a public setting.

2.3. Compute

A Raspberry Pi 4 or 5 is the more sensible default: it is inexpensive to replace if damaged or stolen, has a reasonable power (relevant if the venue only offers a single outlet) consumption, and is already familiar hardware. A specialized device such as a mini PC only becomes worth the extra cost and heat if the game itself needs more graphical power than a Pi can give.

2.4. Software

2.4.1. Game / Platform Layer

Custom game software will be made using Godot as the engine, exported to Raspberry Pi OS's native ARM64 Linux build (2.3). The games will be presented as small minigames, high-rhythm and fast-paced, aimed at maximum enjoyment within a short session.

The whole cabinet experience, not just the minigames, runs as a single Godot application built around a state machine: Idle/Attract, Payment Wait, Gameplay, Results, then back to Idle. Keeping this as one running application rather than separate programs is what makes the idle screen work cleanly, covered in 2.4.3.

2.4.2. Payment & Logging Layer

Payment is QRIS only, no coin path. QRIS needs to be dynamic rather than a single static printed code, since a static code cannot tell the machine which specific payment belongs to which specific play session; it can only confirm that some payment for some amount arrived at some point. A dynamic code generated per session, carrying its own order ID and amount, solves that.

3. Timeline

Phase 1: Prove the software works (3 months, September 2026 - December 2026)

- a. Build one representative minigame in Godot, running on a normal laptop or desktop.
- b. Build the Idle, Payment Wait, Gameplay, Results state machine (2.4.1, 2.4.3), with the payment step mocked out for now.
- c. Done when: the full loop plays start to finish on ordinary hardware and feels good, without needing real money to test it.

Phase 2: Prove the payment integration works (2 months, January 2027 - March 2027)

Register an individual Midtrans account with a KTP and get sandbox access.

- a. Build a small standalone script or app that creates a dynamic QRIS charge, shows the QR, and polls status until paid (2.4.2), kept separate from the game for now.
- b. Test it with a few real, small transactions.
- c. Done when: you can generate a QR, pay it with your own phone, and reliably see the status flip to paid within a few seconds.

Phase 3: Merge software and payment on the real hardware (2 months, April 2027 - May 2027)

- a. Buy the Pi 4/5 (2.3).
- b. Wire the payment flow from Phase 2 into the state machine from Phase 1, replacing the mocked step.
- c. Add the arcade controls (USB encoder, joystick, buttons).
- d. Done when: the full loop, idle, pay with a real QR, play with real controls, results, runs sitting on a table, no cabinet yet.

Phase 4: Build the cabinet (1.5 months, June 2027 - July 2027)

- a. Finalize dimensions around the chosen screen and the parts from Phase 3 (2.2).
- b. Work with a carpenter on the frame, including the access panel and ventilation.
- c. Mount and wire everything inside once the frame is ready.
- d. Done when: the same working loop from Phase 3 runs inside the finished cabinet, powered from a single outlet.

Phase 5: Pilot deployment (2 months, August 2027 - September 2027)

- a. Pick one venue for a trial run, ideally one you can check on often.
- b. Agree with the venue and the chosen cause on how funds are split, handled, and reported.

- c. Watch it under real public use and fix whatever people find that testing did not.
- d. Done when: it has run for a couple of weeks of real public use without needing constant intervention.

Phase 6: Formalize and consider scaling (1 month, October 2027)

- a. Set up the recurring public report, pulling from the payment gateway's ledger and the local session log (2.4.2).
- b. Decide whether to keep it at one venue or add a second, and what should change the second time around.
- c. Done when: a donation report has actually been published at least once, and the scaling decision has been made deliberately rather than by default.